
5장. 프로세스 동기화

목 차

배경

임계구역 문제

임계구역 SW 해결법 (피터슨 해결안)

임계구역 HW 해결법 (동기화 하드웨어)

 세마포

고전적 동기화 문제

모니터

1. 배경

병행 프로세스의 세가지 문제

결정성

상대적인 실행 속도에 관계없이 항상 같은 결과를 도출 여부

상호 배제와 동기화

상호 배제: 어떤 자원에 대해 한 번에 한 프로세스만이 접근

동기화: 프로세스의 진행이 다른 프로세스의 진행에 의존 → 다른 프로세스도 있을 수 있음

교착

프로세스들이 서로 다른 프로세스가 점유한 자원을 요구하여, 아무도 진행되지 못하는 상태

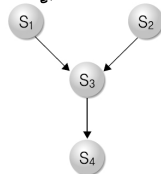
● 결정성(determinacy)

- 프로세스들간의 실행 순서는 독립적 또는 선행 관계
- 선행 관계는 선행 그래프로 표현 가능

알고리즘 4-1 간단한 산술 연산을 수행하는 알고리즘

```
01  a := x + y  → S1
02  b := z + 1  → S2
03  c := a - b  → S3
04  w := c + 1  → S4
```

→ S3이 끝난 후 S4로 진행



[알고리즘4-1]에 대한 선행 그래프

- (1) **c := a - b;** - a와 b가 값을 할당받기 전에 수행하면 안 된다.
- (2) **w := c + 1;** - c 값을 계산하기 전에 수행할 수 없다.
- (3) **a := x + y, b := z + 1;** - 서로 독립적이므로 동시에 수행할 수 있다.

순서 독립적이면 병행성 가능

- 상호배제 (Mutual Exclusion)

- 특정 공유 자원을 한 순간에 한 개의 프로세스만 사용할 수 있는 경우
- 프로세스 하나가 공유 데이터에 접근하는 동안, 다른 프로세스가 해당 데이터를 접근할 수 없게 하는 것
- 교착상태와 기아상태가 발생하는 원인

- 프로세스들간 동기화(synchronization)

- 비동기 프로세스의 대기 시점을 결정

상호배제 및 동기화의 예

● 제한 버퍼의 생산자/소비자 문제

- 버퍼를 모두 사용하기 위해 정수변수 **counter**를 추가
- 새로운 요소 추가 시 **counter**는 1씩 증가, 빼낼 시 1씩 감소
- 동시에 수행될 때 기능이 다르게 동작 가능: **counter = 5** 일 때
 - ‘**counter++**’과 ‘**counter--**’ 동시 수행 시, **counter** 결과 값은 4, 5, 6

↑ 다른 코드가 위해 인해 양방향에 따라 값이 변함.

↓ 동시변수

→ 동시 (동시개념서) 수행시
값의 현상 발생

• 생산자 프로세스

```
item nextProduced;

while (1) {
    while ( counter == BUF_SIZE ) ;
    buffer[in] = nextProduced;
    in = (in + 1) % BUF_SIZE;
    counter++;
}
```

공격
필

• 소비자 프로세스

```
item nextConsumed;

while (1) {
    while ( counter == 0 ) ;
    nextConsumed = buffer[out];
    out = (out+1) % BUF_SIZE;
    counter--;
}
```

공격
필

← wait(mutex);
← signal(mutex);

* **counter := counter + 1**을 기계어로 다음과 같이 작성

register1 := counter;

register1 := register1 + 1;

counter := register1;

$R_1 = \text{Counter}$
 $R_1 = R_1 + 1$
 $\text{Counter} = R_1$

인제/국한(공변, 불변, 해생제)

라운드 3번
↑

어디서 등록 다른 프로세스를 수행하는데
따라 값이 4, 5, 6이 됨

* **counter := counter - 1**을 기계어로 다음과 같이 작성

register2 := counter;

register2 := register2 - 1;

counter := register2;

$R_2 = \text{Counter}$
 $R_2 = R_2 - 1$
 $\text{Counter} = R_2$

T1 : 생산자가 **register1 := counter**를 수행 (**register1 = 5**)

T2 : 생산자가 **register1 := register1 + 1**을 수행 (**register1 = 6**)

T3 : 소비자가 **register2 := counter**를 수행 (**register2 = 5**)

T4 : 소비자가 **register2 := register2 - 1**을 수행 (**register2 = 4**)

T5 : 생산자가 **counter := register1**을 수행 (**counter = 6**)

T6 : 소비자가 **counter := register2**를 수행 (**counter = 4**)

경쟁 상태

임계구역에 대해 상호배제
메시지를 만들지 않으면
경쟁상태 발생

● 경쟁 상태 (Race Condition)

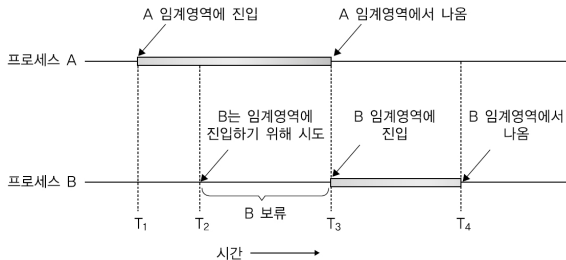
- 공유 데이터에 최종적 결과를 보장할 수 없는 상황
- 여러 프로세스가 공유 데이터를 동시에 접근할 때, 공유 데이터에 대한 접근 순서에 따라 실행 결과가 달라지는 상황
- 최종 값은 어느 프로세스가 마지막으로 끝냈는가에 영향

● 경쟁 상태 방지를 위한 병행 프로세스들의 동기화 실행 방법

- 임계 영역
 - 공유 변수를 어느 한 순간에 한 프로세스만 조작할 수 있도록 함
- 상호 배제
 - 예에서 counter를 조작하는 부분을 임계영역으로 설정하여 상호 배제

2. 임계구역 문제

- 임계구역 또는 임계영역 (**Critical Section**)
 - 한 순간에 하나의 프로세스만 사용할 수 있는 영역
 - 한 프로세스가 임계영역에 들어가면 다른 모든 프로세스는 임계영역으로의 진입 금지
- 진입 상호 배제 및 출구 상호 배제
 - 프로세스 하나가 임계 영역에 있으면 다른 프로세스가 임계 영역에 진입하는 것을 방지
 - 임계 영역을 떠나는 프로세스는 출구 상호배제를 수행, 다른 프로세스의 임계 영역 진입을 허용



임계 영역을 이용한 상호 배제

```
while(TRUE) {
```

```
    진입영역
```

```
    임계영역;
```

```
    출구영역
```

```
    잔류영역;
```

```
}
```

병행 프로세스에서 영역 구분

● 임계영역을 해결하기 위한 세 가지 만족 요건

- **상호 배제:** 한 프로세스가 임계영역에서 수행 중일 때, 다른 프로세스는 임계영역에서 수행할 수 없음
- **진행:** 임계영역에서 실행되는 프로세스가 없는 상태에서 임계 구역으로 진입하려는 프로세스들이 있다면, 다음에 임계 구역으로 진입할 수 있는 프로세스들의 선택을 미루면 안 됨
- **한계 대기:** 한 프로세스가 임계영역을 요청한 후 요청이 수락되기까지 다른 프로세스가 임계영역에 진입할 수 있는 회수를 제한해야 함

↳ 세 가지

이제부터
이제부터

3. 임계구역 SW 해결법

→ 직접 코드를 짜서 해결해나 (3만 하면
1.2는 해결X)

- 2개 프로세스를 위한 임계구역 SW 해결법

- 알고리즘 1 실패 (프로세스 2m)

- 공유 변수 사용: `int turn` (초기값 0)
 - `turn = i` P_i 는 임계 구역에 들어갈 수 있음

- Process P_i

```
do {  
    while (turn != i);  
    critical section  
    turn = i;  
    remainder section
```

```
} while (1);
```

- 상호 배제는 충족하나 진행을 충족하지 못함

- P_1 이 임계구역에 진입하려는데, P_0 가 잔류구역 수행하고 있으면 P_1 은 임계 구역에 진입할 수 없음

● 알고리즘 2 실패

- 공유 변수들: **boolean flag[2]** (초기값 **flag[0]=flag[1]=false**)
 - flag[i] = true** $\square$ P_i 는 임계 구역에 들어갈 준비

• Process P_i

```
do {  
    flag[i] := true;  
    while (flag[j]) ;           critical section  
    flag[i] = false;  
    remainder section  
} while (1);
```

- 상호 배제는 충족하나 진행은 충족하지 못함
 - P_0 와 P_1 이 거의 동시에 임계구역 진입 시도할 때, $flag[0]$ 와 $flag[1]$ 이 둘 다 true 가 되는 경우

- 알고리즘 3 (피터슨의 해결안)

- 알고리즘 1과 2의 공유 변수들을 결합

- Process P_i

```
do {  
    flag[i] := true;  
    turn = i;  
    while ( flag[j] and turn = j );  
    critical section  
    flag[i] = false;  
    remainder section  
} while (1);
```

- 모든 세 조건들을 만족
- 2개 프로세스의 임계영역 문제 해결

↳ 한때 : 2개 미량 해결 X

N 프로세스에 대한 임계구역 해결

임계구역 문제를 SW 적으로

7개까지 해결

● 1974년에 Lamport가 제안한 Bakery 알고리즘

- 낮은 번호를 가진 고객이 먼저 온 것이며 먼저 서비스. 번호표의 가장 낮은 번호는 1. 번호 0은 번호표를 받지 않음을 나타냄.
- 프로세스가 비동기적으로 수행되므로 두 개 이상의 프로세스가 같은 번호표를 받을 수 있음. 이때는 고객의 이름을 대조, 낮은 이름순으로 서비스.
- 프로세스들은 상호 배제, 한계 대기, 진행 조건을 모두 만족.
- 들어온 순서대로 수행되어야 한다는 법칙은 적용되지 않음.

빵집 알고리즘

```
while (1) {  
    ...  
    choosing[i] = true;  
    number[i] = max(number[0], number[1], ..., number[n-1]) + 1;  
    choosing[i] = false;  
    for (j = 0; j < n; j++) { // 모든 프로세스에 대한 번호표 비교루프  
        while (choosing[j]); // Pj가 번호표 받을 때까지 대기  
        while (number[j] < (number[i], i));  
    }  
    // Critical Section  
    ...  
    number[i] = 0;  
    ...  
}
```

choosing[n]; // false로 초기화
number[n]; // 0으로 초기화

4. 임계구역 HW 해결법

- 하드웨어 명령어를 이용한 임계영역 문제 해결
 - 하나의 기억장치 사이클에서 수행
 - **TestAndSet():** 원소적으로 단어의 내용을 테스트하고 수정하는 명령
 - **Swap():** 원소적으로 두 변수를 swap하는 명령
- TestAndSet

```
do {  
    while ( TestAndSet(lock) );  
    critical section  
    lock = false;  
    remainder section  
}
```

→ 매번 loop에 끝날때마다 끝까지 수행

→ True → 기다림
false → 사용

while ();
→ 아무것도 안하면 기다리게 함. (논비)

→ 시간이 짧아야 함 → 코드가 짧음

● Swap

```
do {  
    lock = true;           lock = true ⇒ key = true  
    key = true;           lock = false ⇒ lock = false  
    while (key == true)  
        Swap(lock, key);  
    critical section  
    lock = false;  
    remainder section  
}
```

● 임계 구역에서는 인터럽트 금지 → 객관식↑

- 진입 코드에서 인터럽트를 해제하고 출구 코드에서 복귀
- 다른 프로세스가 오랜 기간 동안 블록될 수 있음

● 임계 구역으로 들어갈 때 바쁜 대기(busy waiting) 유발

- 프로세스를 블록시키고 CPU를 다른 프로세스에게 배정

5. 세마포(Semaphore)

- 기존 상호배제 해결법들은 복잡한 문제에 일반화가 어려움
 - Dijkstra가 제시한 세마포어가 일반화된 도구로 사용
- 바쁜 대기(바쁜 대기)를 요구하지 않는 동기화 도구
- 임계구역 진입의 불공정성 해결

- 세마포어 S – 정수 변수
 - 초기화한 후 두 개의 원소 연산으로만 접근 가능

wait(S) or P(S):

$S--;$

while $S < 0$ do no-op;

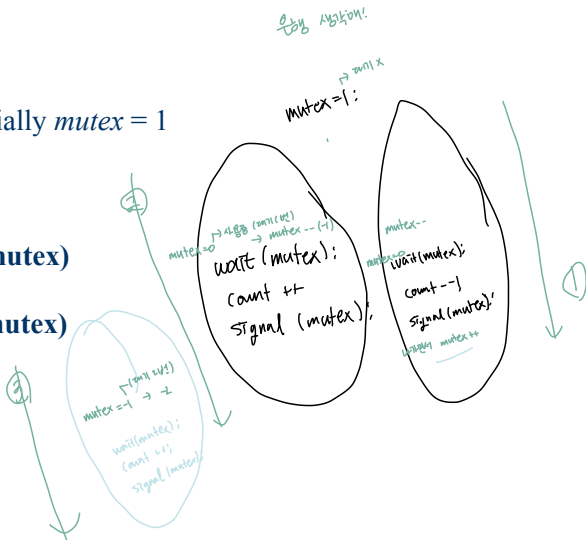
signal(S) or V(S):

$S++;$

N 개의 프로세스들의 임계 구역

- semaphore mutex; //initially $mutex = 1$

```
do {
    wait(mutex);      // P(mutex)
    critical section
    signal(mutex);    // V(mutex)
    remainder section
} while (1);
```



세마포 구현

- 세마포를 레코드로 정의

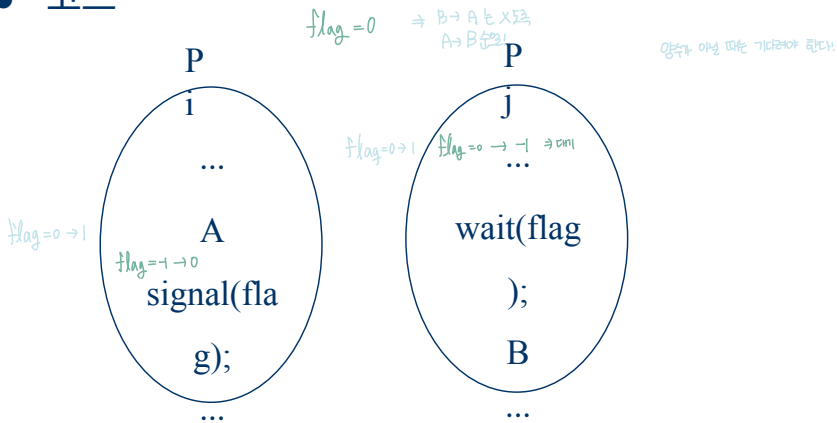
```
typedef struct {  
    int value;  
    struct process *L;  
} semaphore;
```

- 세마포 연산들

```
wait(S):  
    S.value--;  
    if (S.value < 0) {  
        add this process to S.L;  
        block;  
    }  
signal(S):  
    S.value++;  
    if (S.value <= 0) {  
        remove a process P from S.L;  
        wakeup(P);  
    }
```

동기화 도구로서의 세마포

- P_i 에서 A 를 실행한 이후에만 P_j 에서 B 를 실행
- 0으로 초기화된 세마포어 $flag$ 사용
- 코드



교착상태와 기아상태

- 교착상태(Deadlock)

- 둘 이상의 프로세스들이 대기중인 프로세스들 중 단지 하나에 의해서만 야기될 수 있는 사건을 무한정 기다리는 상황
- S 와 Q 가 1로 초기화된 세마포인 경우

$P0$ $P1$

$wait(S);$ $wait(Q);$

$wait(Q);$ $wait(S);$

□

□

$signal(S);$ $signal(Q);$

$signal(Q)$ $signal(S);$

- 기아상태(Starvation)

- 무한 정지(indefinite blocking) 프로세스들이 세마포어 큐에서 제거되지 않고 무한정 대기하는 것

6. 고전적 동기화 문제

● 한정 버퍼 문제 (생산자/소비자 문제)

- 공유 데이터: semaphore full=0, empty=n, mutex=1;
- empty는 비어 있는 버퍼의 개수, full은 차 있는 버퍼의 개수
- mutex는 상호배제를 위한 세마포

• 생산자 프로세스

```
do {  
    ... // nextp에 하나 생산  
    wait(empty);  
    wait(mutex);  
    ... // nextp를 버퍼에 추가  
    signal(mutex);  
    signal(full);  
} while (1);
```

• 소비자 프로세스

```
do {  
    wait(full);  
    wait(mutex);  
    ... // 버퍼에서 하나 제거  
    signal(mutex);  
    signal(empty);  
    ... // 소비  
} while (1);
```

Readers-Writers 문제

공유 데이터: semaphore mutex=1, wrt=1;

- writer 프로세스

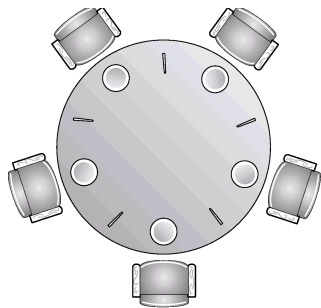
```
wait(wrt);  
...  
writing is performed  
...  
signal(wrt);
```

- reader 프로세스

```
wait(mutex);  
readcount++;  
if (readcount == 1)  
    wait(wrt);  
signal(mutex);  
...  
reading is performed  
...  
wait(mutex);  
readcount--;  
if (readcount == 0)  
    signal(wrt);  
signal(mutex);
```


식사하는 철학자 문제

공유 데이터: semaphore chopstick[5] = {1, 1, 1, 1, 1};



철학자 i

```
o {  
    wait(chopstick[i]); // 왼쪽 젓가락을 집음  
    wait(chopstick[(i+1) % 5]); //오른쪽 젓가락을 집음  
    ...  
    eat  
    ...  
    signal(chopstick[i]); // 왼쪽 젓가락을 놓음  
    signal(chopstick[(i+1) % 5]); //오른쪽 젓가락을 놓음  
    ...  
    think  
    ...  
} while (1);
```

7. 모니터 (Monitors)

- 많은 세마포 사용시, 잘못된 사용으로 오류가 발생하기 쉬움

- 모니터

- 프로그래밍 언어에서 제공되는 프로그램
래머 정의 연산자들의 집합

- 한 순간에 하나의 프로세스만 모니터 안에서
활동하도록 보장

```
monitor monitor-name
{
    shared variable declarations
    procedure body P1 (...) {
        ...
    }
    procedure body P2 (...) {
        ...
    }
    procedure body Pn (...) {
        ...
    }
    initialization code (...) {
        ...
    }
}
```

● 조건 변수 (condition variable)

- 한 프로세스가 모니터 내에서 대기하도록 하는 데 사용

- condition x, y ;

- 오직 wait와 signal 동작 operation과 함께 사용

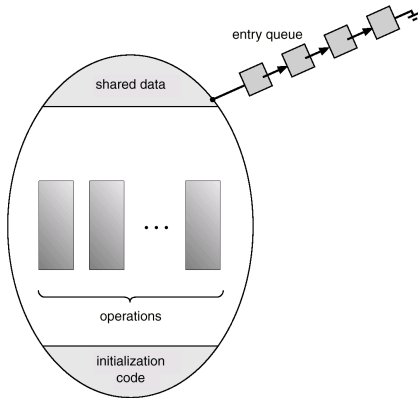
- $x.wait()$

- 이 동작을 호출한 프로세스는 또 다른 프로세스가 $x.signal()$ 을 호출할 때까지 대기

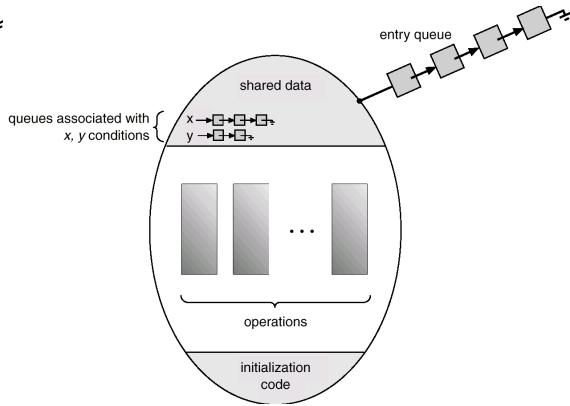
- $x.signal()$

- 정확하게 하나의 대기하고 있는 프로세스를 재실행

- 대기하고 있는 프로세스가 없으면 아무 효과가 없음



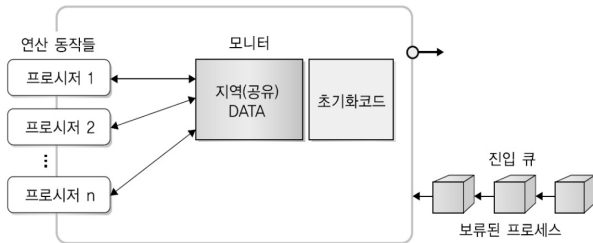
모니터 구조



조건 변수를 가지는 모니터

● 모니터의 구조

- 하나 이상의 프로시저와 초기화 코드, 공유 데이터로 구성된 소프트웨어 모듈
- 모니터 내의 프로시저는 모니터 내에 정의된 변수와 형식 매개변수들만 접근
- 모니터 경계에서 한 번에 한 프로세스만 진입하도록 제어, 상호 배제를 지킴
 - 모니터 내의 프로세스는, 사용하고자 하는 자원이 다른 프로세스에 이미 할당되었으면 대기할 수 있음
 - 프로그래머는 상호 배제를 위한 동기화 제약 조건을 명시적으로 코드화할 필요 없음



모니터의 일반 구조

- 다음 모니터는 이진 세마포어의 기능을 가짐.
- `acquire()`는 `wait()`처럼 동작하고, `release()`는 `signal()`처럼 동작.

자원 할당 모니터
<pre> monitor resourceallocator { bool busy; condition x; acquire() //자원할당받기전 wait() { if (busy) x.wait(); busy = true; } release() //자원 반납후 signal() { busy = false; x.signal(); } resourceallocator() // 초기화 루틴 { busy = false; } } </pre>

resourceallocator R;

...

R.acquire(); // 자원 할당

...

// 자원 사용

R.release(); // 자원 반납

...

자원 할당 모니터

원형버퍼 모니터

```
monitor ringbuffermonitor
{
    item buffer[slots];
    int slotinuse, nextfill, nextempty;
    condition HasData, HasSpace;

    fillaslot(item s) //생 산자
    {
        if (slotinuse == slots) HasSpace.wait();
        buffer[nextfill] = s;
        slotinuse++;
        nextfill = (nextfill+1)%slots;
        HasData.signal();
    }
    emptyaslot(item &s) //소비자
    {
        if (slotinuse == 0) HasData.wait();
        slotdata = buffer[nextempty];
        slotinuse--;
        nextempty = (nextempty+1)%slots;
        HasSpace.signal();
    }
    ringbuffermonitor() // 초기화 루틴
    {
        slotinuse = nextfill = nextempty = 0;
    }
}
```

```
ringbuffermonitor buf;
...
buf.fillaslot(input);
...
buf.emptyaslot(&output);
...
```

모니터를 이용한 식사하는 철학자 해결안

monitor DP

```
{  
    enum{THINKING,HUNGRY,EATING} state[5];  
    condition self[5];
```

```
void pickup (int i) {  
    state[i] = HUNGRY;  
    test(i);  
    if (state[i] != EATING) self[i].wait;  
}
```

```
void putdown (int i) {  
    state[i] = THINKING;  
    // test left and right neighbors  
    test((i + 4) % 5);  
    test((i + 1) % 5);  
}
```

```
void test (int i) {  
    if ( (state[(i + 4) % 5] != EATING)  
        && (state[i] == HUNGRY) &&  
        (state[(i + 1) % 5] != EATING) ) {  
        state[i] = EATING ;  
        self[i].signal () ;  
    }  
}
```

```
initialization_code() {  
    for (int i = 0; i < 5; i++)  
        state[i] = THINKING;  
}
```